

Model Evaluation & Optimization



Outline

- Classification Model Evaluation
- The Bias–Variance Tradeoff
- Hyperparameter Tuning Loop
- Reducing Model Complexity
(e.g., Tree Pruning)

Classification Model Evaluation

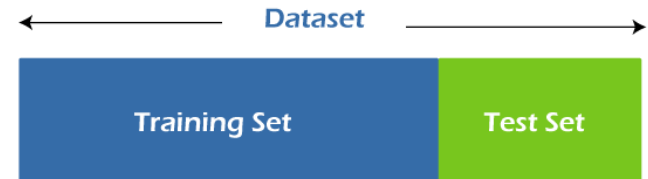


This part is adapted from this [source](#)

Defining Training and Test Set Splits

Training Set (70–80%):

- Used to **learn patterns** and adjust model parameters



Test Set (20–30%)

- Never used in training
- Measures true performance on unseen data

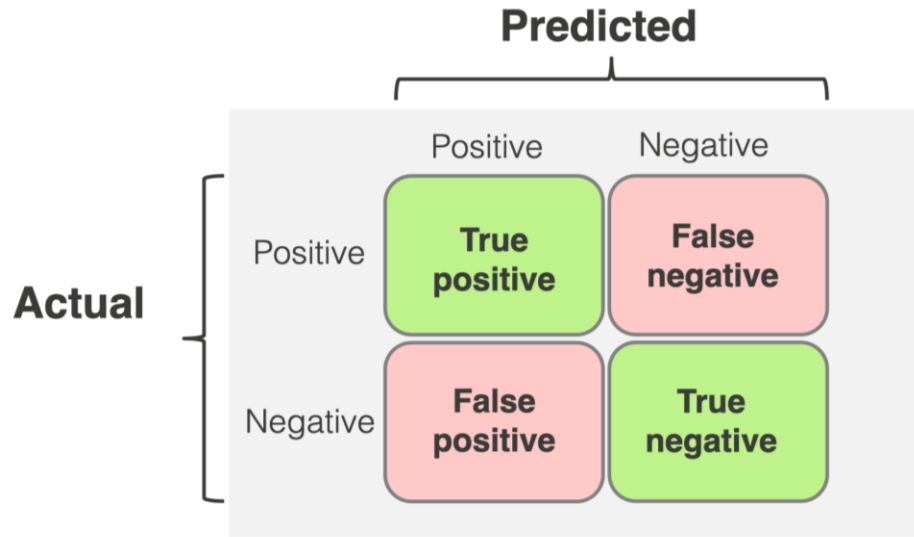
Why Split the Data?

- Avoid **data leakage**
- Get an **unbiased** model evaluation
- Detect **overfitting vs. underfitting**
- Enable fair model comparison

Evaluating Classifier Accuracy

- **What to Evaluate?**

- **Accuracy** measures how often the classifier is correct
- **Errors** and correct predictions are summarized using a confusion matrix
- A confusion matrix shows:



- ✓ **True Positives (TP):** correctly predicted positive cases
- ✓ **True Negatives (TN):** correctly predicted negative cases
- ✗ **False Positives (FP):** “false alarms” — predicted positive but actually negative
- ✗ **False Negatives (FN):** “missed cases” — predicted negative but actually positive

- Using TP, TN, FP, and FN, we compute key evaluation metrics such as:
Accuracy, Precision, Recall, F1-Score

Confusion Matrix Example 🐱



Cat	Not Cat	Cat	Not Cat	Cat	Not Cat	Cat	Not Cat	Cat	Cat
-----	---------	-----	---------	-----	---------	-----	---------	-----	-----

✅ Compute Evaluation Metrics

1. **Accuracy:** How often the model is correct.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{5 + 2}{5 + 2 + 1 + 1} = \frac{7}{10} = 0.70$$

➡ **The model is correct 70% of the time** → Reasonable but hides important details

2. **Precision** 🎯 (How many predicted cats are real cats?)

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{5}{5 + 1} = 0.83$$

%83 ➡ **of the images predicted as "Cat" were actually Cats** → High Precision

3. **Recall** 🔍 (How many actual cats were found?)

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{5}{5 + 2} = 0.71$$

➡ **The model detected 71% of all actual cats** → Moderate Recall → It misses some actual cats (false negatives)

4. **F1-Score** ⚖️ (Balance of Precision & Recall)

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \cdot \frac{0.83 \times 0.71}{0.83 + 0.71} = 0.76$$

➡ **Good overall balance between precision and recall** (useful when both FP and FN matter)

	Predicted	
	Cat	Not Cat
Actual Cat	TP = 5	FN = 2
Actual Not Cat	FP = 1	TN = 2

False Positive vs False Negative

	Predicted	Actual	Correct?	
1.	Not fraud	Not fraud	✓	True Negative
2.	Not fraud	Not fraud	✓	True Negative
3.	Not fraud	Fraud	✗	False Negative
4.	Fraud	Fraud	✓	True Positive
...				
n.	Fraud	Not fraud	✗	False Positive

- There are two types of errors the model can make:

✗ **False Positive (FP) – “False Alarm”** 🚨 → inconvenience, unnecessary alerts, wasted resources

The model predicts **Fraud / Spam / Disease**, but the actual case is **Normal**.

- Example: Labeling a normal transaction as fraudulent.

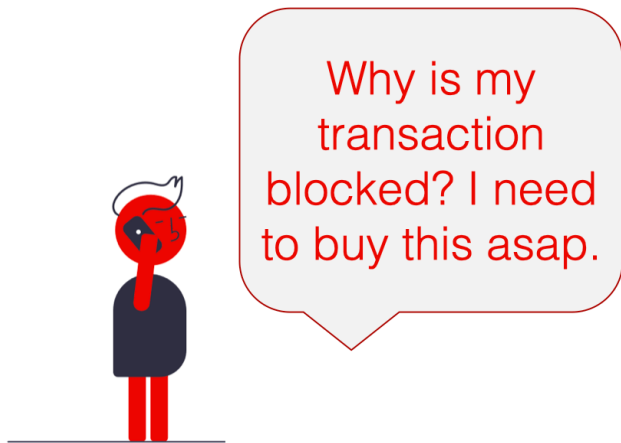
✗ **False Negative (FN) – “Missed Case”** ⚠️ → dangerous or costly misses (often more harmful)

The model predicts **Normal**, but the actual case is **Fraud / Spam / Disease**.

- Example: Missing a truly fraudulent transaction.

False Positive vs False Negative

- Two types of classification errors - each with different real-world consequences



False positive



False negative

False Positive (FP) — “False Alarm”

The model predicts **Positive** but the actual case is **Negative**

- Flagging a normal transaction as fraud
- Marking a legitimate email as spam

Impacts:

- Annoyance, unnecessary actions
- Wasted resources

False Negative (FN) — “Missed Case”

The model predicts **Negative** but the actual case is **Positive**

- Missing a truly fraudulent transaction
- Failing to detect a real disease

Impacts:

- Often **more harmful**
- Critical cases go undetected

Spam Classifier

📧 True Positive (TP) – Correctly Detected Spam

- Spam emails **correctly classified** as spam

✓ 600 spam emails

📧 True Negative (TN) – Correctly Classified as Non-Spam

- Legitimate emails **correctly classified** as not spam

✓ 9000 non-spam emails

• 🚨 False Positive (FP) – False Alarm:

- Legitimate emails **incorrectly classified** as spam

✗ **More harmful** → blocking a legitimate email can be costly

✗ 100 legitimate emails marked as spam

• ⚠️ False Negative (FN) – Missed Spam:

- Spam emails **incorrectly** delivered to the inbox

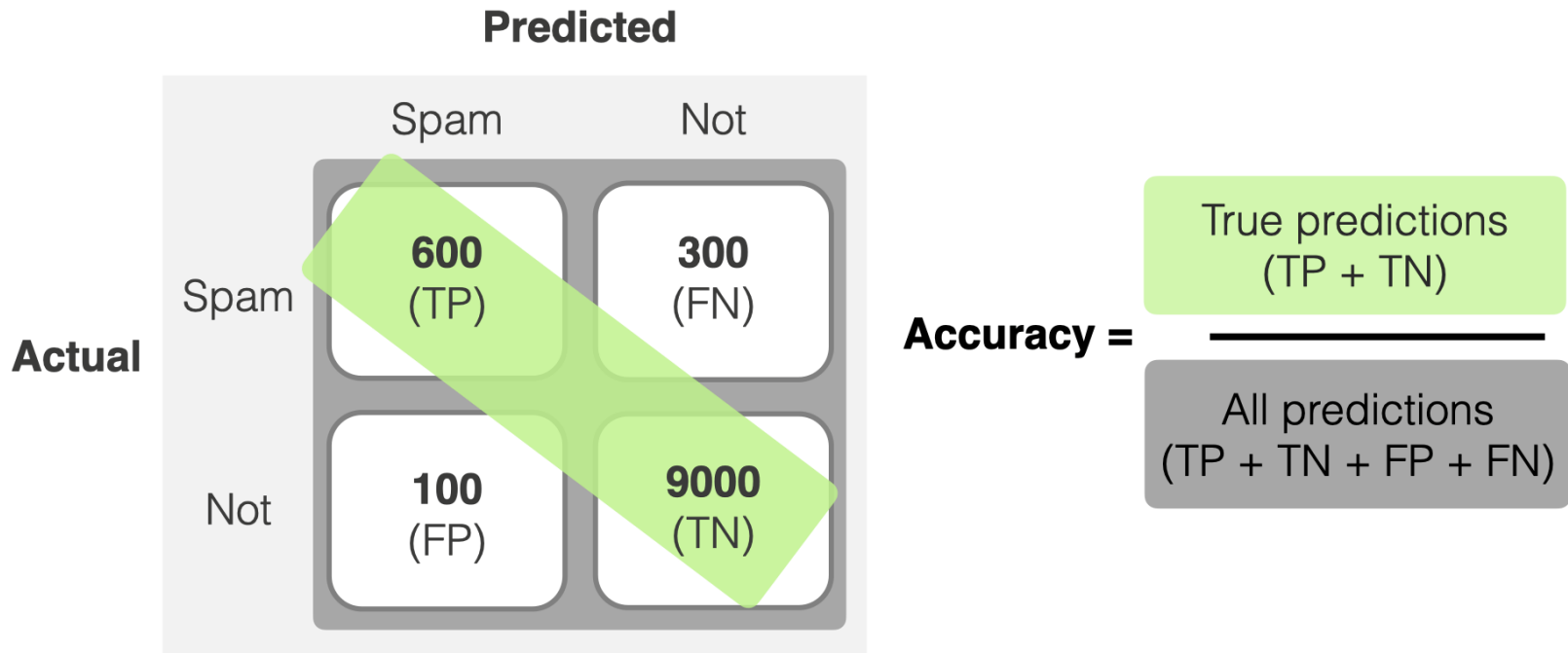
✗ Inconvenience → spam reaches the user

✗ 300 spam emails not detected



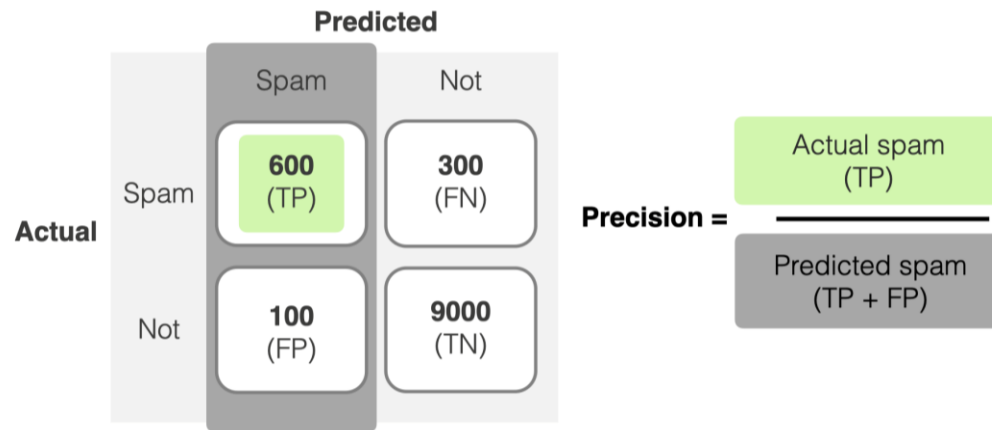
		Predicted	
		Spam	Non-spam
Actual	Spam	600 (True positive)	300 (False negative)
	Non-spam	100 (False positive)	9000 (True negative)

Accuracy



- Accuracy measures how often the model makes correct predictions
- 👉 **Accuracy** = $(9000 + 600) / 10000 = 0.96 \rightarrow 96\%$ ✓ The model was correct in 96% of cases
- However, accuracy can be misleading for imbalanced datasets when one class is much more frequent
 - e.g., 9100 normal emails vs. 900 spam
 - High accuracy may hide poor performance on the minority class

Precision



- **Precision answers:** When the model predicts Spam, **how often is it correct?**
 - It measures the proportion of predicted positives that are actually positive
 - Precision = $600 / (600 + 100) = 0.86$ → 86% of emails classified as spam were truly spam
 - High precision means **few false positives** 🚨
- Prioritize precision when **false positives are costly**, such as:
 - 📧 Spam filtering — avoid blocking important emails
 - 🏭 Manufacturing Quality Control — flagging a perfectly good batch of expensive components (like microchips) as defective, leading to massive financial waste
 - ✈️ Airport Security — flagging a harmless everyday item as a dangerous weapon, resulting in flights cancellations

Recall

		Predicted	
		Spam	Not
Actual	Spam	600 (TP)	300 (FN)
	Not	100 (FP)	9000 (TN)

Recall

$$\frac{\text{Actual spam (TP)}}{\text{All spam (TP + FN)}}$$

- Recall answers: Out of all actual Spam emails, **how many did the model correctly detect?**
 - It measures the proportion of actual positives that are correctly identified
 - $\text{Recall} = 600 / (600 + 300) = 0.67$ → 67% of actual spam emails were correctly detected. The other 33% reached the inbox
 - High recall means **few false negatives** ⚠️
- Prioritize recall when **false negatives are more harmful**, such as:
 - 🔬 Medical diagnosis — avoid missing real disease cases
 - 💳 Fraud detection — avoid missing fraudulent transactions
 - 🔥 Safety-critical alerts — avoid missing fires, intrusions, or system failures

F1 Score

$$\text{F1 Score} = \frac{2 \times (\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$$

- F1 Score answers: How well does the **model balance Precision and Recall**?
 - F1 score is the harmonic mean of precision and recall
 - ➡ F1 Score ≈ 0.75 → the classifier has a reasonable balance between avoiding false alarms and catching spam
 - Higher F1 means both Precision and Recall are reasonably high
- Prioritize F1 Score when both **false positives and false negatives matter**, especially with imbalanced datasets
 - ✉ Spam detection — balance blocking bad emails and preserving good ones
 - 💳 Fraud detection — balance catching fraud and avoiding false alerts
 - 🏥 Medical diagnosis — balance detecting real cases and limiting false alarms

Evaluating Classifier Accuracy

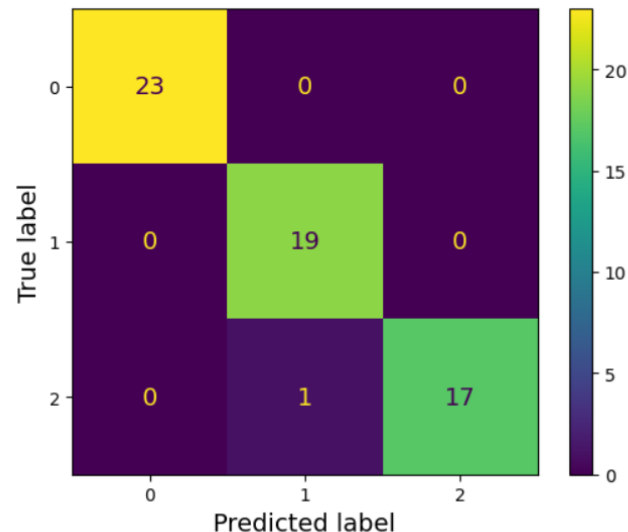
```
► y_pred = tree_clf.predict(X_test)
print("Predicted Labels:", y_pred) #Predicted labels the testing points
print("True Labels:      ", y_test) #True labels
print("Testing Accuracy:", metrics.accuracy_score(y_test, y_pred)) #1 mistake .. 59/60 = 0.983
```

```
Predicted Labels: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0 1 1 2 1 2 1 2 1 0 2 1 0 0 0 1]
True Labels:      [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0 1 2 2 1 2 1 2 1 0 2 1 0 0 0 1]
Testing Accuracy: 0.9833333333333333
```

```
► from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, y_pred) # 1 wrong prediction is made .. Accuracy |
```

```
|: array([[23,  0,  0],
        [ 0, 19,  0],
        [ 0,  1, 17]], dtype=int64)
```

```
► from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
cm = confusion_matrix(y_test, y_pred) # 1 wrong prediction is made .. Accuracy
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=tree_clf.classes_)
disp.plot()
plt.show()
```



Evaluating Classifier Accuracy

- **How to Evaluate?**

- Normally, a separate independently sampled test set is used to measure accuracy of a classification model
- Evaluation methods:
 - **Holdout Method**: divide data set (e.g., 70/30) as training and test sets

```
In [6]: ▶ #Take 40% of the data as testing and the remaining 60% as training
X_train, X_test, y_train, y_test = train_test_split(X_iris, y_iris, test_size=0.4, random_state=42)
y_train = y_train.to_numpy()
y_test = y_test.to_numpy()
print("Training Data", X_train.shape)
print("Testing Data", X_test.shape) #60 points for testing

Training Data (90, 2)
Testing Data (60, 2)
```

Decision tree trained on X_train

```
In [7]: ▶ tree_clf = DecisionTreeClassifier(random_state=42)
tree_clf.fit(X_train, y_train)
# Accuracy on the training dataset using the score method
print("Training Accuracy:", tree_clf.score(X_train, y_train))

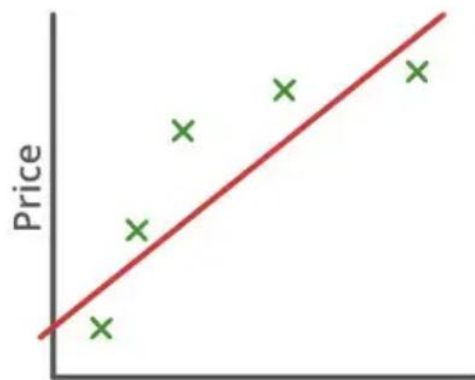
Training Accuracy: 0.9888888888888889
```

Making Prediction

```
In [8]: ▶ y_pred = tree_clf.predict(X_test)
print("Predicted Labels:", y_pred) #Predicted labels the testing points
print("True Labels:      ", y_test) #True labels
print("Testing Accuracy:", metrics.accuracy_score(y_test, y_pred)) #1 mistake .. 59/60 = 0.983

Predicted Labels: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0 1 1 2 1 2 1 2 1 0 2 1 0 0 0 1]
True Labels:      [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0 1 2 2 1 2 1 2 1 0 2 1 0 0 0 1]
Testing Accuracy: 0.9833333333333333
```

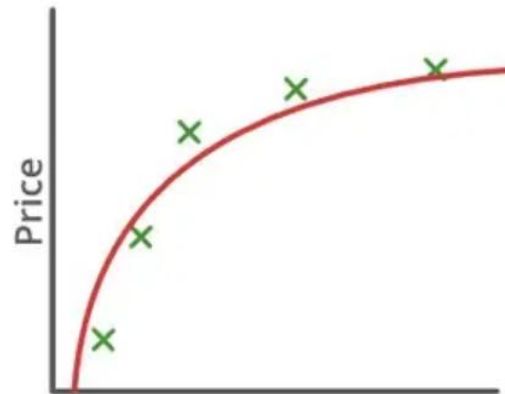
Bias–Variance Tradeoff



Size

$$\theta_0 + \theta_1 x$$

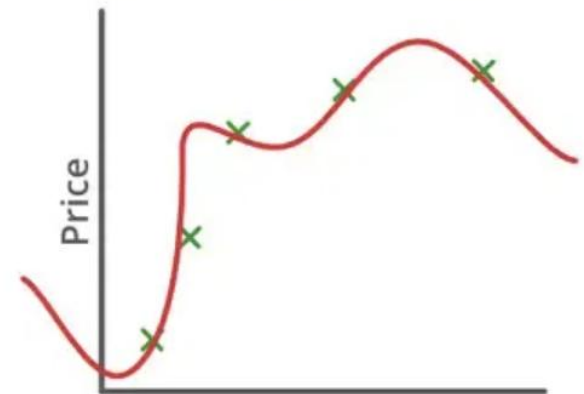
High Bias
(Underfitting)



Size

$$\theta_0 + \theta_1 x + \theta_2 x^2$$

Low Bias, Low Variance
(Goodfitting)



Size

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

High Variance
(Overfitting)



The Bias–Variance Tradeoff

Underfitting (High Bias)

- Model too **simple** to capture patterns in data
- Poor on both training and test data

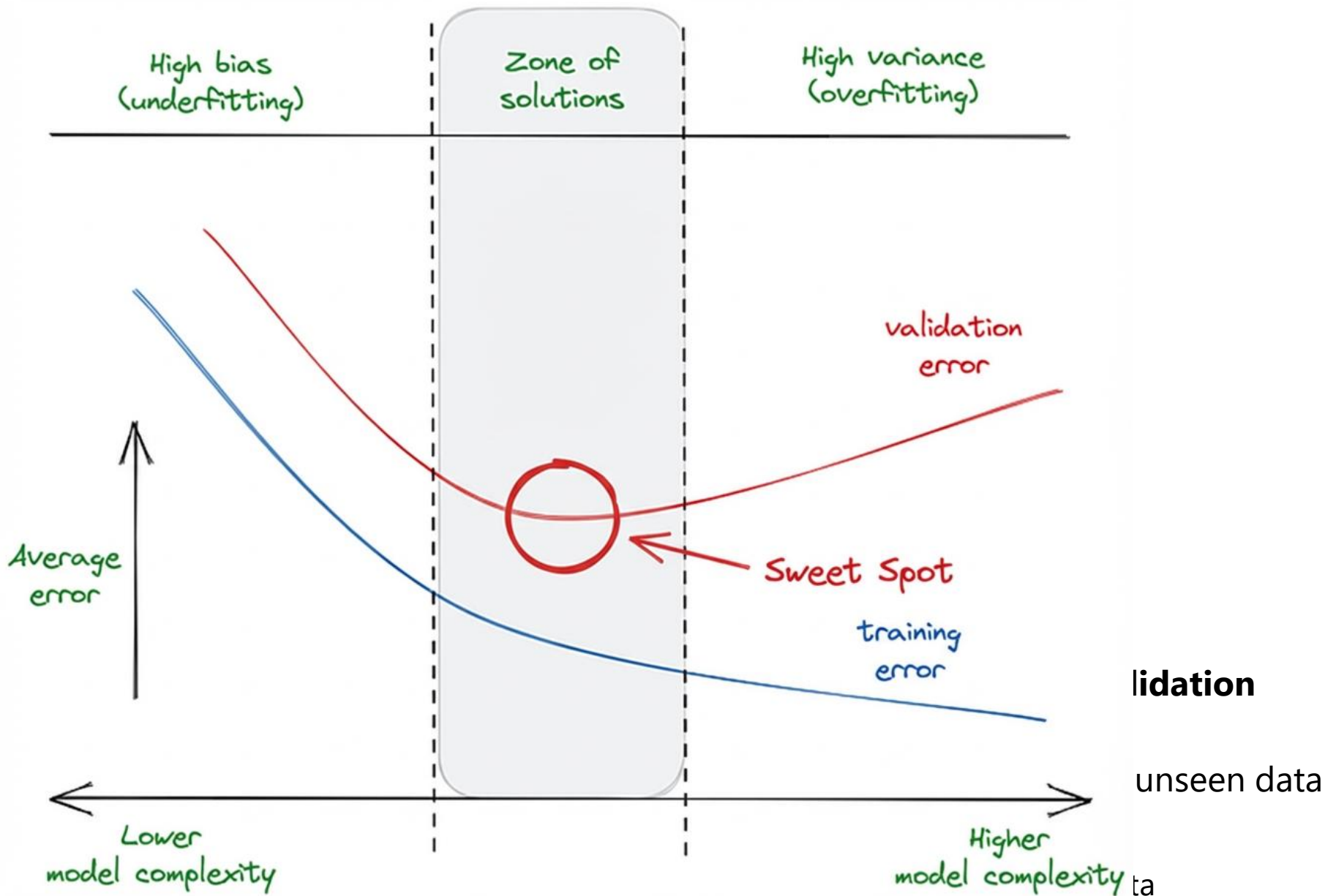
Overfitting (High Variance)

- Model too **complex** — memorizes noise
- Great on training data, poor on test data

The Sweet Spot

- **Goal:** find complexity that minimizes **total error** on unseen data
- **Hyperparameter tuning** is *how* we navigate this tradeoff

The Sweet Spot



Strategies to Reduce High Bias (Underfitting)

Increase Model Complexity

- Hyperparameter-tuning (e.g., find optimal max_depth) to find the right model—the sweet spot between bias and variance
- E.g., use deeper decision trees, more neural network layers,

Choose a **More Expressive Model**

- Move from linear → polynomial models
- Use ensembles (Random Forest, Gradient Boosting) instead of simple models

 **Train Longer** (for iterative models): Increase training iterations

 Feature Engineering: Add informative features

Strategies to Handle High Variance (Overfitting)

Simplify the Model

- Prune decision trees (reduce unnecessary branches)
- Limit model capacity: Decrease `max_depth`, increase `min_samples_split`
- Reduce number of parameters (for NNs: fewer layers/neurons)

Apply Regularization

- L1/L2 regularization → penalize large weights
- Dropout → randomly turn off neurons to prevent memorization
- Early stopping → stop training when validation error begins to rise

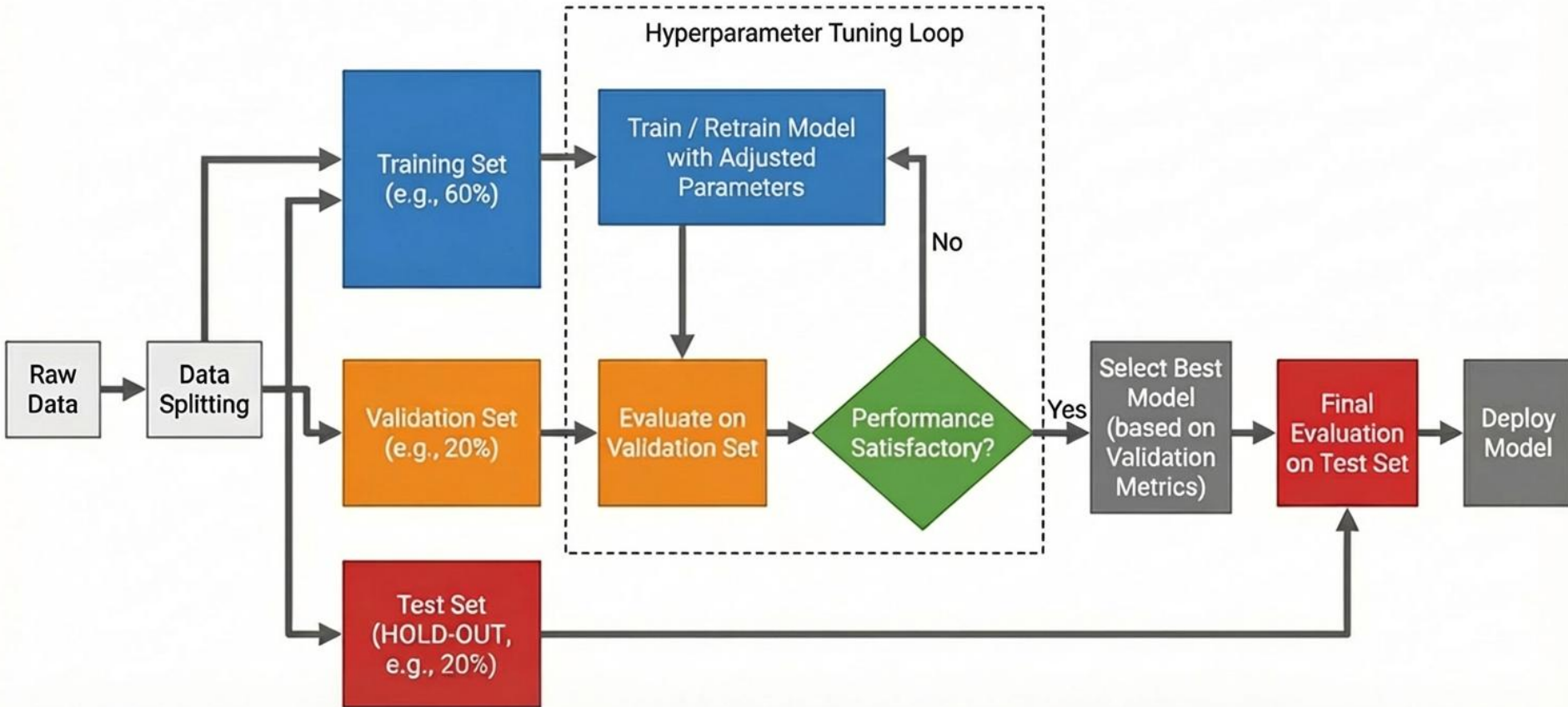
3. Increase Training Data to give the model a better understanding of the true patterns

- Collect more data or use **data augmentation** (especially in vision tasks)

4. Cross-validation (e.g., K-Fold) to help detect models that generalize poorly across folds

5. Use Ensemble Methods: Bagging (e.g., Random Forest) reduces variance by averaging many models

Hyperparameter Tuning Loop



Golden Rule: Splitting Your Data Correctly

Tune hyperparameters on the validation set, we split the dataset into three distinct buckets:

1. Training Set (e.g., 70%)

- Used to learn patterns from this majority chunk of data

2. Validation Set (e.g., 15%)

- Practice exam to test our model during development, tune its settings, and select the best-performing version
- Enables diagnosing weak spots of the model and tune its parameters

3. Test Set (e.g., 15%)

- Final exam. Used at the very end to evaluate how the model will perform on new, unseen data



Cross-Validation — Why & How

- Problem with a Single Validation Split
 - Results depend on **which** samples land in the validation set
 - Small datasets amplify this randomness
- K-Fold Cross-Validation
 1. Split data into **K** equal folds
 2. For each fold *i*: train on all folds except *i*, validate on fold *i*
 3. Average the K scores → more **robust** estimate

Cross Validation

Every single point is used for testing once

```
In [12]: from sklearn.model_selection import cross_val_score
scores = cross_val_score(tree_clf, X_iris, y_iris, cv=5)
print(scores)

print("%.2f accuracy with a standard deviation of %.2f" % (scores.mean(), scores.std()))

[0.96666667 0.96666667 0.9        0.96666667 1.        ]
0.96 accuracy with a standard deviation of 0.03
```

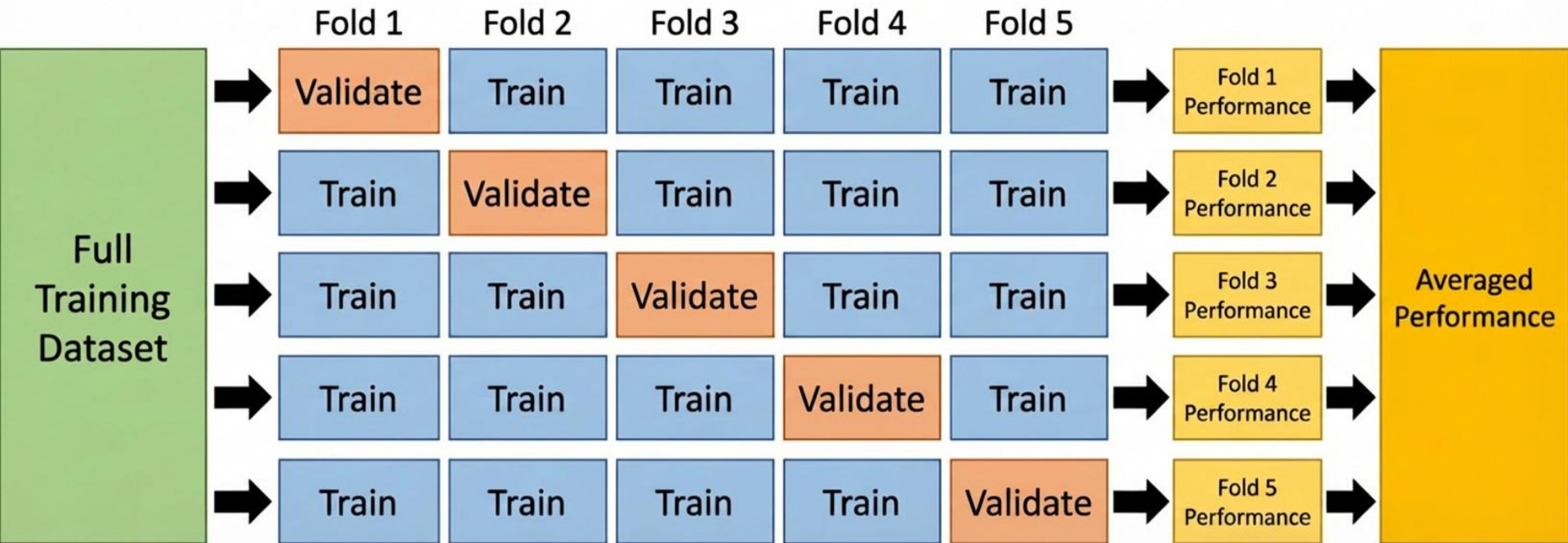
```
In [18]: #50 cross validation. Each time, train on 147 and test on 3 samples
scores = cross_val_score(tree_clf, X_iris, y_iris, cv=50)
print(scores.round(2))

print("%.2f accuracy with a standard deviation of %.2f" % (scores.mean(), scores.std()))

[1.  1.  1.  1.  1.  1.  0.67 1.  1.  1.  1.  1.  1.  1.
 1.  1.  1.  1.  1.  0.67 0.67 1.  1.  1.  1.  1.  0.67 0.67
 1.  0.67 1.  1.  1.  0.67 1.  1.  1.  1.  0.67 1.  1.  1.
 1.  1.  1.  1.  1.  1.  1.  1. ]
0.95 accuracy with a standard deviation of 0.12
```

K-Fold Cross-Validation

K-Fold Cross-Validation (K=5)



Splitting up the dataset into smaller chunks and rotating through them as training sets



Decision Tree — Key Hyperparameters

Hyperparameter	What It Controls	Typical Range
<code>max_depth</code>	Maximum tree depth	1 – 20+
<code>min_samples_split</code>	Min samples to split a node	2 – 50
<code>min_samples_leaf</code>	Min samples in a leaf	1 – 30
<code>max_features</code>	Features considered per split	"sqrt", "log2", int, float
<code>criterion</code>	Split quality metric	"gini", "entropy", "log_loss"
<code>max_leaf_nodes</code>	Max number of leaves	5 – 100+



Grid Search — Exhaustive Tuning

- Define a **grid** of hyperparameter values

max_depth	min_samples_split	criterion
2, 4, 6, 8, 10	2, 5, 10, 20	gini, entropy

→ $5 \times 4 \times 2 = 40$ combinations $\times$ 5 folds = **200 model fits**

- Train & evaluate **every combination** via cross-validation
- Select the combination with the best CV score

✓ Pros	✗ Cons
Guaranteed to find the best in the grid	Computationally expensive
Simple to implement and understand	Scales poorly
Reproducible results	



Grid Search — Custom Implementation

```
import numpy as np
from itertools import product
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold

X, y = load_iris(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Define the hyperparameter grid
param_grid = {
    "max_depth": [2, 4, 6, 8, 10],
    "min_samples_split": [2, 5, 10],
    "criterion": ["gini", "entropy"],
}

best_score, best_params = 0, {}

# Iterate over every combination in the grid
keys = list(param_grid.keys())
for values in product(*param_grid.values()):
    params = dict(zip(keys, values))
    clf = DecisionTreeClassifier(random_state=42, **params)
    score = cross_val_score(clf, X, y, cv=cv,
        scoring="accuracy").mean()

    if score > best_score:
        best_score, best_params = score, params

print(f"Best params: {best_params}")
print(f"Best CV accuracy: {best_score:.3f}")
```



Grid Search — Scikit-Learn GridSearchCV

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold

X, y = load_iris(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

param_grid = {
    "max_depth": [2, 4, 6, 8, 10],
    "min_samples_split": [2, 5, 10],
    "criterion": ["gini", "entropy"],
}

grid_search = GridSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_grid=param_grid,
    cv=cv,
    scoring="accuracy",
    n_jobs=-1,          # use all CPU cores
    return_train_score=True,
)
grid_search.fit(X, y)

print(f"Best params: {grid_search.best_params_}")
print(f"Best CV accuracy: {grid_search.best_score_:.3f}")

# The best model is already refit on the full training data
best_model = grid_search.best_estimator_
```

Attribute	Description
<code>.best_params_</code>	Best hyperparameter combination
<code>.best_score_</code>	Mean CV score of the best combo
<code>.best_estimator_</code>	Model refit with best params on all data
<code>.cv_results_</code>	Full results dict (convertible to DataFrame)



Random Search — Smarter Sampling

- Instead of every combination, **sample** N random configurations
- Each hyperparameter is drawn from a **distribution** or a list
- Often finds good results with **far fewer evaluations**

Why Random Search Works

- Many hyperparameters have **low importance** — Grid Search wastes evaluations on unimportant dimensions
- Random Search **spreads** evaluations more efficiently across the important dimensions

When to Prefer Random Search

- Large hyperparameter space (≥ 3 hyperparameters)
- Continuous hyperparameters (e.g., learning rate, regularization)
- Limited computational budget



Random Search — Scikit-Learn

RandomizedSearchCV

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold
from scipy.stats import randint, uniform

X, y = load_iris(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Distributions for continuous/discrete sampling
param_distributions = {
    "max_depth": randint(2, 20),                # uniform integer [2, 20)
    "min_samples_split": randint(2, 30),
    "min_samples_leaf": randint(1, 20),
    "criterion": ["gini", "entropy", "log_loss"],
    "max_features": uniform(0.3, 0.7),          # float in [0.3, 1.0)
}

random_search = RandomizedSearchCV(
    estimator=DecisionTreeClassifier(random_state=42),
    param_distributions=param_distributions,
    n_iter=100,                                # number of random samples
    cv=cv,
    scoring="accuracy",
    n_jobs=-1,
    random_state=42,
)
random_search.fit(X, y)

print(f"Best params: {random_search.best_params}")
print(f"Best CV accuracy: {random_search.best_score_:.3f}")
```



Analyzing Tuning Results

```
import pandas as pd

# Convert grid/random search results to a DataFrame
results = pd.DataFrame(random_search.cv_results_)

# Top 5 configurations by mean test score
top5 = (results[["params", "mean_test_score",
                "std_test_score", "mean_fit_time"]]
        .sort_values("mean_test_score", ascending=False)
        .head(5))
print(top5.to_string(index=False))

# Examine how max_depth affects performance
depth_results =
(results.groupby("param_max_depth")["mean_test_score"]
 .agg(["mean", "max"])
 .sort_index())
print(depth_results)
```



Detecting Overfitting During Tuning

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold

X, y = load_iris(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

param_grid = {"max_depth": list(range(1, 21))}

grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv=cv,
    scoring="accuracy",
    return_train_score=True,    # key flag
)
grid.fit(X, y)

# Compare train vs. validation scores
import pandas as pd
df = pd.DataFrame({
    "max_depth": range(1, 21),
    "Train Acc": grid.cv_results_["mean_train_score"],
    "Val Acc": grid.cv_results_["mean_test_score"],
})
print(df.to_string(index=False))
# Look for: growing gap between Train and Val → overfitting
```

Red Flags

- **Train accuracy ≈ 1.0** while Val accuracy plateaus or drops
- **Large gap** between train and validation scores
- **High std** across folds → unstable model



End-to-End Example — Full Pipeline

```
from sklearn.datasets import load_wine
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import (
    train_test_split, GridSearchCV, StratifiedKFold
)
from sklearn.metrics import classification_report

# Step 1: Load data and hold out test set
X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# Step 2: Define hyperparameter grid
param_grid = {
    "max_depth": [3, 5, 7, 10, 15],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 3, 5],
    "criterion": ["gini", "entropy"],
}

# Step 3: Run Grid Search with stratified 5-fold CV
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv=cv,
    scoring="accuracy",
    n_jobs=-1,
    return_train_score=True,
)
grid.fit(X_train, y_train)

# Step 4: Inspect tuning results
print(f"Best params: {grid.best_params}")
print(f"Best CV accuracy: {grid.best_score_:.3f}")

# Step 5: Final unbiased evaluation on held-out test set
y_pred = grid.best_estimator_.predict(X_test)
print("\n--- Test Set Classification Report ---")
print(classification_report(y_test, y_pred))
```



Multiple Metrics Tuning

```
from sklearn.datasets import load_breast_cancer
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

param_grid = {
    "max_depth": [3, 5, 7, 10],
    "min_samples_leaf": [1, 3, 5, 10],
}

# Optimize for multiple metrics; select best by F1
grid = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid,
    cv=cv,
    scoring={"accuracy": "accuracy", "f1": "f1", "roc_auc": "roc_auc"},
    refit="f1", # refit best model using F1 as primary metric
    return_train_score=True,
    n_jobs=-1,
)
grid.fit(X, y)

print(f"Best params (by F1): {grid.best_params_}")
print(f"Best CV F1: {grid.best_score_:.3f}")
```

When to Use

- **Imbalanced data** → optimize F1 or AUC, not accuracy
- **Multi-objective** → track several metrics, pick primary via `refit`



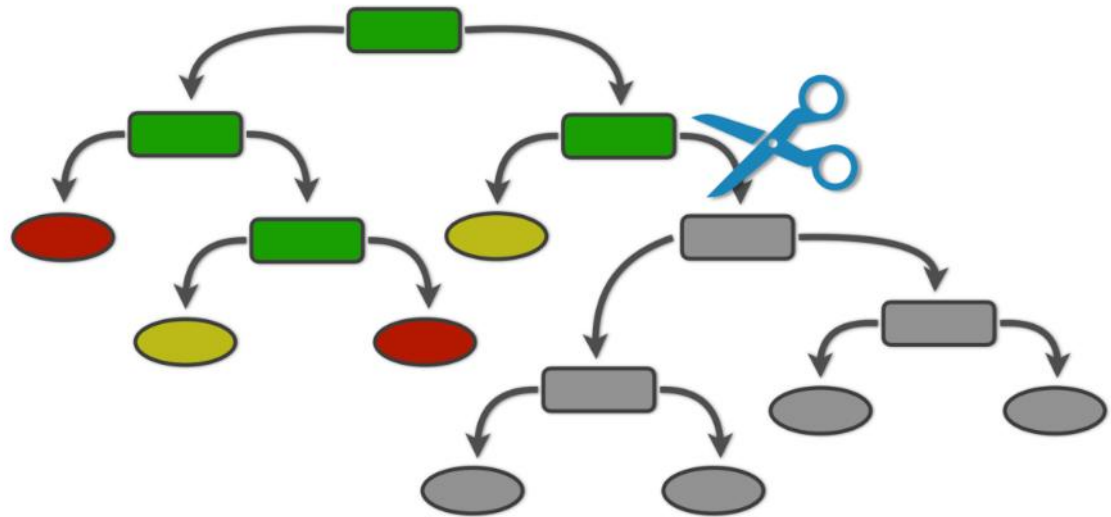
Summary

What We Covered

1. **Parameters vs. Hyperparameters** — learned vs. set by the engineer
 2. **Validation Set** — dedicated data for tuning (never the test set)
 3. **K-Fold Cross-Validation** — robust, averaged evaluation
 4. **Grid Search** — exhaustive; best for small spaces
 5. **Random Search** — efficient; best for large or mixed spaces
 6. **Decision Tree Tuning** — `max_depth`, `min_samples_split`, `min_samples_leaf`, `criterion`
 7. **Complete Workflow** — split → define → search → inspect → test
- Hyperparameter tuning is **systematic experimentation**:
 - Use cross-validation, automate with scikit-learn, and always respect the train/validation/test boundary.



Tree Pruning



Pruning DT

- *Overfitting* the training data can be reduced by pruning the tree
- Two pruning strategies:
 - ***Pre-pruning***: stop building the tree before it is "complete"
 - ***Post-pruning***: build a full tree and then “cut back” certain subtrees and replace them with leaf nodes
- Pre-pruning is much faster but can stop prematurely, post-pruning is more accurate

Pre-pruning strategies

Stop building the tree:

- At **max** tree depth
- When **number of instances in a leaf** is less than certain minimum
- When **percentage of majority class** at a node is more than $n\%$
- When the change in Error rate, Gini index, or Entropy is below some threshold



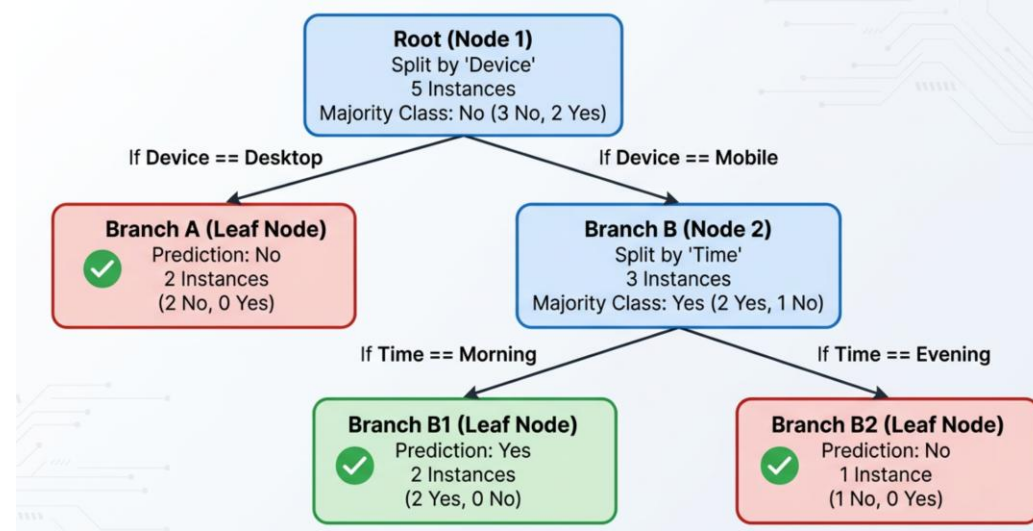
Reduced Error Pruning (REP)

- A post-pruning technique that evaluates the effect of removing subtrees using a validation set
- For every internal node (starting from the bottom-up):
 1. **Replace the subtree with a leaf** 
 - The leaf predicts the **majority class** among training samples reaching that node
 2. **Evaluate on validation set**
 - If pruning **reduces errors** or **does not increase** them  keep the prune 
 - Otherwise, **revert** back to the original subtree
 3. **Repeat**
 - Continue pruning nodes until no further improvement is possible

Reduced Error Pruning - Example

Training set (Used to build the tree)

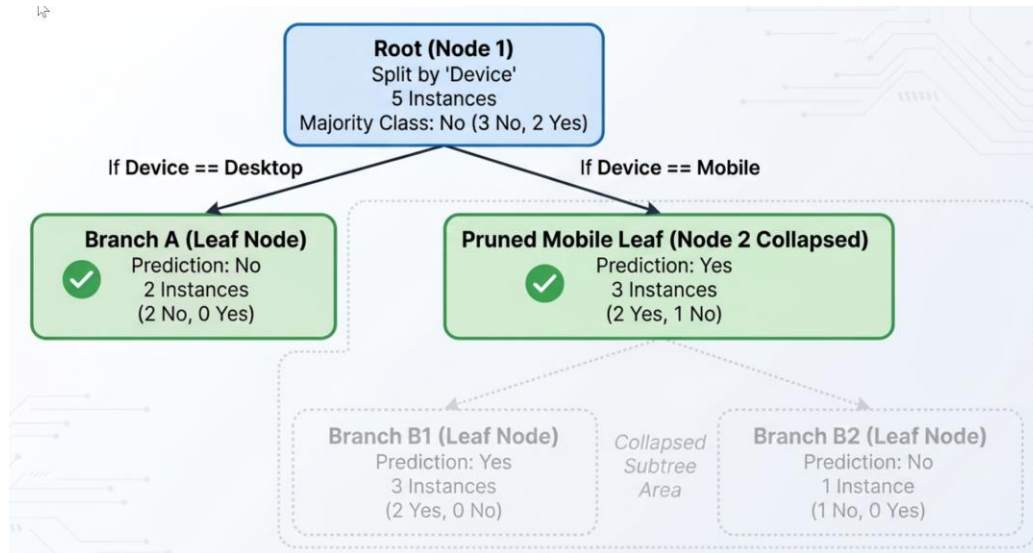
ID	Device	Time	Clicked Ad?
1	Mobile	Morning	Yes
2	Mobile	Morning	Yes
3	Mobile	Evening	No
4	Desktop	Morning	No
5	Desktop	Evening	No



- We prune and evaluate the **lowest internal node**, Node 2 (the split on Time for Mobile devices)

Validation set (Used to evaluate pruning)

ID	Device	Time	Clicked Ad?
V1	Mobile	Morning	No
V2	Mobile	Morning	No
V3	Mobile	Evening	Yes
V4	Desktop	Morning	No



Reduced Error Pruning - Example

- We prune and evaluate the **lowest internal node**, Node 2 (the split on Time for Mobile devices)
- We compare the **validation errors** of the Unpruned Tree versus a Pruned Tree where Node 2 is collapsed into a leaf predicting its training majority class ("Yes")

Validation ID	Features (Device, Time)	Actual Target	Unpruned Prediction	Unpruned Result	Pruned Prediction	Pruned Result
V1	Mobile, Morning	No	Yes	Error	Yes	Error
V2	Mobile, Morning	No	Yes	Error	Yes	Error
V3	Mobile, Evening	Yes	No	Error	Yes	Correct!
Total Errors				3		2

Because the number of errors on the validation dataset decreased from 3 to 2, we permanently prune Node 2

